STOCK PRICE TREND PREDICTION

Machine Learning Based Predictive Analytics Project

Nathan Kennedy, Batyr Rasulov, Alejandro Barranco-Leyte
Course: SWE - 452
Date: 2/15/2026

Table of Contents

1. **Executive Summary**
2. **Introduction**
3. **Problem Statement**
4. **Objectives and Goals**
5. **Scope of Work**
6. **Technical Approach**
7. **Risk Assessment and Management**
8. **Budget and Resource Allocation**
9. **Design and Requirements Specification**
    - **9.1 Purpose and Audience**
    - **9.2 Definitions, Acronyms, and Abbreviations**
10. **System Overview**
11. **Functional Requirements**
    - **11.1 Data Input Requirements**
    - **11.2 ML Model Requirements**
    - **11.3 Cloud Service Requirements**
12. **Non-Functional Requirements**
13. **Data Requirements**
14. **System Architecture**
15. **Training and Evaluation Plan**
16. **Deployment Requirements**
17. **User Interface Requirements**

Executive Summary

The Stock Price Trend Prediction project aims to develop a machine learning–based system capable of predicting short-term stock market trends by identifying whether a stock's price will move upward or downward on the following trading day. Rather than attempting to forecast exact stock prices—which is highly unreliable due to market volatility—this project focuses on trend classification to provide more realistic and interpretable results.

Using historical stock market data and technical indicators such as moving averages, volatility, and returns, the system applies supervised machine learning models to identify directional patterns. The project delivers value by supporting data-driven insights for educational and analytical purposes while maintaining ethical standards and realistic expectations. This solution is scalable, cost-effective, and aligned with modern financial analytics practices.

Introduction

Financial markets are complex systems influenced by economic indicators, investor sentiment, geopolitical events, and unforeseen external factors. As a result, accurately predicting exact stock prices is extremely challenging and often impractical. This project addresses this challenge by focusing on stock price trend prediction rather than precise price forecasting.

The scope of this project involves analyzing historical stock price data and applying machine learning classification techniques to determine future price direction. The approach aligns with both business and user needs by providing a simplified, interpretable, and realistic decision-support framework. This project also serves as an educational tool, demonstrating the application of data science and machine learning techniques in financial analytics.

Problem Statement

Stock market prediction is a long-standing challenge due to the complex and stochastic nature of financial markets. Traditional stock prediction models frequently attempt to forecast exact future prices using historical data. However, stock prices are influenced by a wide range of unpredictable factors, including macroeconomic conditions, geopolitical events, investor sentiment, and sudden market news. As a result, price-based prediction models are highly sensitive to market noise, randomness, and volatility, which often leads to unstable and unreliable results.

Many existing approaches suffer from overfitting, where models perform well on historical data but fail to generalize to unseen market conditions. This lack of generalizability can mislead users into placing unwarranted confidence in precise numerical predictions, increasing analytical and decision-making risk. Furthermore, exact price forecasting does not align well with how financial practitioners typically interpret market behavior, as even small prediction errors can result in significant real-world consequences.

There is a need for a more practical and reliable solution that focuses on predicting stock price direction rather than exact values. By addressing this gap, the project aims to provide meaningful insights while minimizing unrealistic expectations and analytical risk.

Objectives and Goals

The objectives of this project are defined using the SMART criteria:

- **Specific:** Predict whether a stock's closing price will increase or decrease on the next trading day.
- **Measurable:** Achieve a classification accuracy between 60% and 65%, which is realistic for financial market prediction.
- **Achievable:** Use historical stock data and established machine learning models such as Logistic Regression and Random Forest.
- **Relevant:** Provide practical insights into market behavior and demonstrate real-world application of machine learning.
- **Time-Bound:** Complete data collection, model development, evaluation, and documentation within one academic term.

Scope of Work

The project scope includes the following phases:

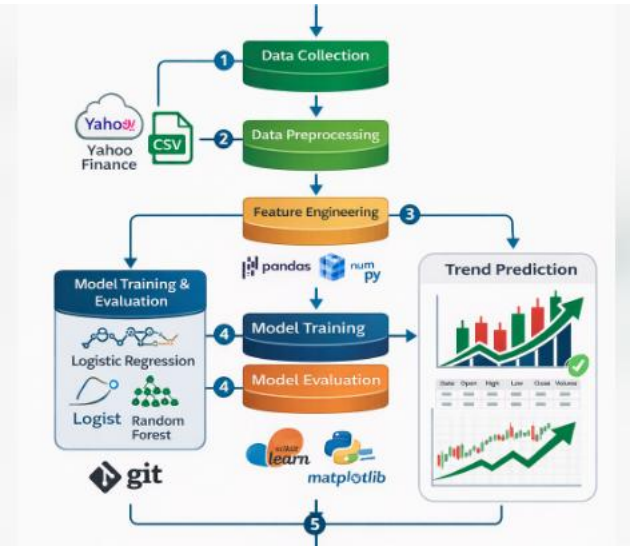
1. **Data Collection:** Acquire historical stock price data (Open, High, Low, Close, Volume) from reliable sources such as Yahoo Finance.
2. **Data Preprocessing:** Handle missing values, normalize data, and prepare datasets for modeling.
3. **Feature Engineering:** Create technical indicators such as moving averages, returns, and volatility.
4. **Model Development:** Train classification models to predict stock price trends.
5. **Model Evaluation:** Assess model performance using accuracy, precision, recall, and confusion matrices.
6. **Visualization:** Present actual versus predicted trends using graphical plots.
7. **Deployment:** Implement a basic dashboard using Streamlit for demonstration purposes.

Deliverables: trained model, evaluation results, visualizations, and a final project report.

Technical Approach

The project follows a modular and scalable architecture. Python is used as the primary programming language due to its extensive ecosystem for data analysis and machine learning. Key libraries include Pandas and NumPy for data manipulation, Scikit-learn for model training, Matplotlib for visualization, and yFinance (Yahoo Finance) for data collection.

The system employs classification-based machine learning models such as Logistic Regression and Random Forest. A time-series-aware train-test split is used to preserve chronological order and prevent data leakage. Version control is managed using Git, ensuring reproducibility and collaborative development.



The overall architecture consists of data ingestion, preprocessing, feature extraction, model training, evaluation, and visualization layers.

Risk Assessment and Management

Several risks were identified during project planning:

Risk	**Impact**	**Mitigation Strategy**
Market Noise	Reduced accuracy	Focus on trend classification instead of price prediction
Overfitting	Poor generalization	Use time-series split and limit model complexity
Data Quality Issues	Biased predictions	Clean and validate historical data
Unrealistic Expectations	Misinterpretation	Include clear disclaimers and explain limitations

Contingency plans include model comparison, feature refinement, and parameter tuning.

Budget and Resource Allocation

The Stock Price Trend Prediction project is designed to be cost-effective by leveraging open-source technologies and readily available academic resources. As a result, the overall financial expenditure for the project is minimal, making it suitable for an educational and research-focused environment.

Software Resources

All software tools used in this project are open-source and freely available, eliminating licensing costs. These include Python for development, Pandas and NumPy for data manipulation, Scikit-learn for machine learning model implementation, Matplotlib for data visualization, and yFinance for historical stock data acquisition. Version control and collaboration are managed using Git and GitHub, which further supports reproducibility and team-based development without additional cost.

Estimated Cost: $0

Data Resources

Historical stock price data is obtained from publicly available sources such as Yahoo Finance through the yFinance library. These data sources are free for academic use and sufficient for trend-based prediction tasks.

Estimated Data Acquisition Cost: $0

Human Resources

Human resources consist of student team members responsible for various project activities, including data collection, feature engineering, model development, performance evaluation, visualization, testing, and documentation. Responsibilities are distributed to ensure efficient use of time and to promote collaborative learning.

Design and Requirements Specification

Purpose and Audience

This document defines the finalized technical design and system requirements for the Stock Price Trend Prediction system. It serves as a comprehensive blueprint for implementation, deployment, and maintenance.

Target Audience:

- Software Developers
- Cloud Engineers
- Project Stakeholders

This document formalizes all functional and non-functional requirements, ML workflows, cloud infrastructure details, deployment strategies, and evaluation criteria.

Definitions, Acronyms, and Abbreviations

Term	Definition
ML	Machine Learning
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
F1 Score	Harmonic mean of precision and recall
RMSE	Root Mean Squared Error
AWS	Amazon Web Services
S3	Simple Storage Service
EC2	Elastic Compute Cloud
IAM	Identity and Access Management
REST	Representational State Transfer
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
GPU	Graphics Processing Unit
SLA	Service Level Agreement

System Overview

The system predicts whether a stock's closing price will increase or decrease the following trading day using supervised classification models.

High-Level Functionality

1. Retrieve historical stock data.
2. Preprocess and engineer features.
3. Train ML classification model.
4. Evaluate model performance.
5. Deploy model to cloud environment.
6. Provide predictions via web dashboard and API.

High-Level Architecture

[Yahoo Finance API]
↓
[Data Ingestion Layer]
↓
[Data Preprocessing + Feature Engineering]
↓
[Model Training Pipeline]
↓
[Model Registry (Cloud Storage)]
↓
[Inference API Service]
↓
[Streamlit Dashboard]

Cloud-hosted components enable scalable model serving and remote access.

Functional Requirements

Data Input Requirements

- Input formats: CSV or JSON
- Required fields: Date, Open, High, Low, Close, Volume
- Time-series-aware split required
- Missing value handling: Forward fill or row removal
- Data normalization: Min-Max scaling or Standardization

ML Model Requirements

- Binary classification (Up/Down)
- Models: Logistic Regression, Random Forest
- Inference latency ≤ 200ms

Cloud Requirements

- AWS EC2 for compute
- AWS S3 for storage
- IAM for access control

Compute Requirements

- Minimum 2 vCPUs
- 8GB RAM
- Optional GPU support for advanced models

API Requirements

- RESTful endpoint
- JSON request/response
- HTTPS encryption

Non-Functional Requirements

Performance: Inference < 200ms

Availability: 99% uptime during demo

Scalability: 100 concurrent users

Security: HTTPS encryption and IAM roles

Data Requirements

Data Sources

- Yahoo Finance API (via yFinance)

Data Format

- CSV format
- Structured time-series data

Expected Dataset Size

- 5–10 years of daily data
- Approx. 2,500 trading records per stock

Storage Needs

- < 1GB for raw and processed datasets
- < 100MB for trained models

Data Privacy

- Public financial data only
- No personal user data collected

Retention Policy

- Retain processed datasets for project duration
- Archive trained models with versioning

Annotation Requirements

- Label = 1 if Close(t+1) > Close(t)
- Label = 0 otherwise

System Architecture

Detailed Architecture Diagram

```
┌──────────────────────┐
│ Yahoo Finance API    │
└──────────────────────┘
           │
           ↓
┌──────────────────────┐
│ Data Ingestion       │
│ (Python + yFinance)  │
└──────────────────────┘
           │
           ↓
┌──────────────────────┐
│ Feature Engineering  │
│ Pandas / NumPy       │
└──────────────────────┘
           │
           ↓
┌──────────────────────┐
│ Model Training       │
│ Scikit-learn         │
└──────────────────────┘
           │
           ↓
┌──────────────────────┐
│ Model Storage (S3)   │
└──────────────────────┘
           │
           ↓
┌──────────────────────┐
│ EC2 Inference API    │
└──────────────────────┘
           │
           ↓
┌──────────────────────┐
│ Streamlit Dashboard  │
│                      │
```

Communication

- API calls over HTTPS
- JSON data exchange
- IAM-based access control

Training and Evaluation Plan

Pipeline: Data ingestion → Cleaning → Feature Engineering → Time-series split → Training → Hyperparameter tuning → Evaluation

Evaluation Metrics:

Accuracy ≥ 60%, F1 Score ≥ 0.58

Deployment Requirements

Cloud Environment Setup

- AWS EC2 instance (Ubuntu)
- Python environment via virtualenv
- Model artifacts stored in S3
- IAM roles configured

Deployment Strategy

- Real-time inference API
- Optional batch retraining weekly

CI/CD Pipeline

- GitHub repository
- Automated testing
- Auto-deploy on merge to main branch

Monitoring

- CloudWatch logging
- Performance metric tracking
- Manual monthly evaluation

User Interface Requirements

Dashboard Requirements

- Built with Streamlit
- Features:
    - Stock ticker input
    - Date range selection
    - Trend prediction display
    - Probability confidence
    - Historical trend visualization

Performance

- Page load time < 3 seconds
- Responsive design

API Documentation

Endpoint:

POST /predict

Request:

```
{
  "ticker": "AAPL",
  "date": "2026-02-10"
}
```

Response:

```
{
  "prediction": "UP",
  "probability": 0.64
}
```